King Mongkut's University of Technology Thonburi
Faculty of Engineering, Department of Computer Engineering
CPE 333 Operating Systems, 1/2024

PROBLEM SESSION Week #3: Process Manipulation and Monitoring.

---

1) Learn the important Linux commands for manipulating and monitoring the process or jobs

1.1) Discuss the difference between "top" and "ps" commands.

```
top - 19:46:31 up 33 min,  1 user,  load average: 0.80, 0.94, 1.09
Tasks: 386 total,   2 running, 383 sleeping,   0 stopped,   1 zombie
%Cpu(s):  3.2 us,  4.8 sy,  0.0 ni, 91.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  15048.3 total,    675.0 free,   5264.0 used,   9045.7 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   9784.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  24224 pai       20   0 2016212 216480 170836 S  60.0   1.4   0:04.98 ptyxis
  27317 pai       20   0   13512   6420   4208 R  40.0   0.0   0:00.30 top
   6533 pai       20   0 9380308 495444 226544 R  20.0   3.2   4:01.16 gnome-shell
      1 root      20   0   27612  18180  11672 S   0.0   0.1   0:05.68 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-kblockd
     11 root      20   0       0      0      0 I   0.0   0.0   0:00.22 kworker/0:1-events
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.19 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:01.45 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kprobe-optimizer
     20 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/2
     23 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/2
     24 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/2
     25 root      20   0       0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/2
     27 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/2:0H-kblockd
     28 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/4
     29 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/4
     30 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/4
     31 root      20   0       0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/4
     33 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/4:0H-kblockd
```

"top" displays data similar to a task manager of all the programs running and system resources in the computer and its live update

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps -l
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000   24257   24231  0  80   0 -  2985 do_wai pts/1    00:00:00 bash
0 S  1000   29157   24257  0  80   0 -  4028 hrtime pts/1    00:00:00 sleep
4 R  1000   29160   24257 99  80   0 -  3200 -      pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps
    PID TTY          TIME CMD
  24257 pts/1    00:00:00 bash
  29157 pts/1    00:00:00 sleep
  29168 pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
pai        24257   24231  0 19:34 pts/1    00:00:00 /usr/bin/bash
pai        29157   24257  0 19:57 pts/1    00:00:00 sleep 1000
pai        29171   24257 99 19:57 pts/1    00:00:00 ps -f
```

"ps" only displays the program running on that exact terminal at the exact time the command

1.2) What is the purpose of the “nice” command? How can you “nice” a process?

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 1 sleep 1000 &
[1] 29482
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 2 sleep 1000 &

[2] 29485
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 4 sleep 1000 &

[3] 29487
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 3 sleep 1000 &

[4] 29489
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps -l
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000   29452   24231  0  80   0 -  2856 do_wai pts/1    00:00:00 bash
0 S  1000   29482   29452  0  81   1 -  4028 hrtime pts/1    00:00:00 sleep
0 S  1000   29485   29452  0  82   2 -  4028 hrtime pts/1    00:00:00 sleep
0 S  1000   29487   29452  0  84   4 -  4028 hrtime pts/1    00:00:00 sleep
0 S  1000   29489   29452  0  83   3 -  4028 hrtime pts/1    00:00:00 sleep
4 R  1000   29493   29452  0  80   0 -  3200 -      pts/1    00:00:00 ps
```

“nice” assigns a nice value or NI to the program running. The program with lower nice value will have priority before the program with higher.

1.3) How to kill the processes and/or jobs? Give an example in the report.
use command: `kill PID`

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 4 sleep 1000 &
[1] 29999
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ nice -n 3 sleep 1000 &
[2] 30003
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps
    PID TTY          TIME CMD
  29452 pts/1    00:00:00 bash
  29999 pts/1    00:00:00 sleep
  30003 pts/1    00:00:00 sleep
  30005 pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ kill 30003
[2]+  Terminated              nice -n 3 sleep 1000
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system$ ps
    PID TTY          TIME CMD
  29452 pts/1    00:00:00 bash
  29999 pts/1    00:00:00 sleep
  30009 pts/1    00:00:00 ps
```

2) Discuss these situations

First situation

1) Create a shell script (just a text file), named ss1_1.sh.
2) In the script, the “sleep” command is called within 10 seconds.
3) Run the script ss1_1.sh (./ss1_1.sh), wait for 10 second
4) Run the script ss1_1.sh again with the “&” at the end of command (./ss1_1.sh &)
5) Discuss the difference between both running methods. (3 and 4)

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./ss1_1.sh
ps
ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  30431 pts/0    00:00:00 bash
  30454 pts/0    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  30431 pts/0    00:00:00 bash
  30460 pts/0    00:00:00 ps
```

with out “&”: the command run in the foreground making me unable to interact what the terminal and the “ps” command only work after the program finish

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./ss1_1.sh &
[1] 30470
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  30431 pts/0    00:00:00 bash
  30470 pts/0    00:00:00 ss1_1.sh
  30472 pts/0    00:00:00 sleep
  30473 pts/0    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  30431 pts/0    00:00:00 bash
  30483 pts/0    00:00:00 ps
[1]+  Done                    ./ss1_1.sh
```

with “&”: the program runs in the background which allows me to still use the terminal; in this case, I use the “ps” command and see while the program is running.

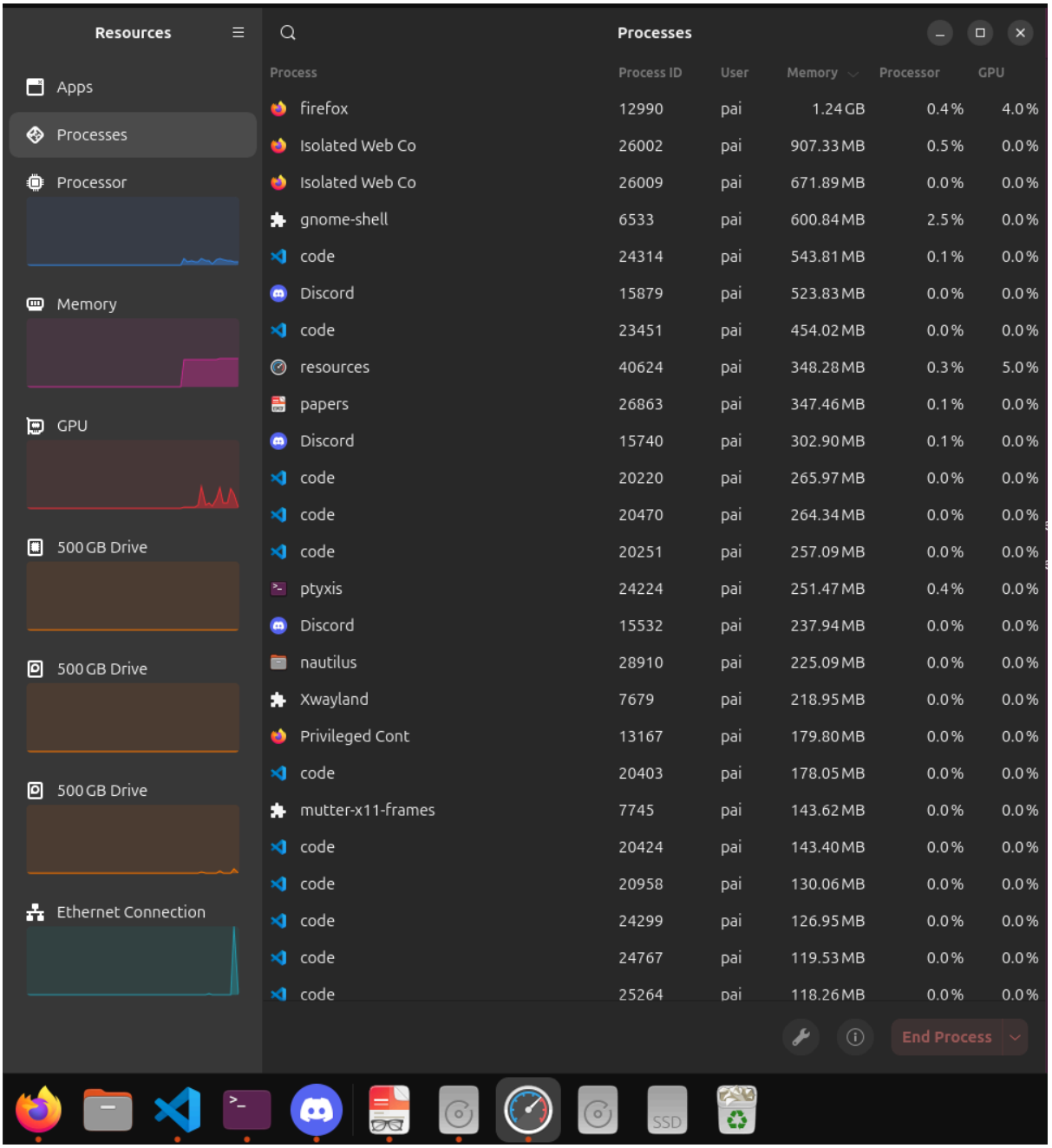
Second situation

1) Create a shell script, named ss1_2.sh.
2) In the script, the “sleep” command is called within 1000 seconds.
3) Run the script ss1_2.sh (./ss1_2.sh), then press CTRL+Z.
4) Use “jobs” command, see the result and discuss the result of pressing CTRL+Z
5) Show how to use “fg” and “bg” commands to resume the script ss1_2.sh and discuss the difference of both resume commands.

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./ss1_2.sh
ps
ps
^Z
[1]+  Stopped                 ./ss1_2.sh
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ jobs
[1]+  Stopped                 ./ss1_2.sh
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  31157 pts/1    00:00:00 bash
  31719 pts/1    00:00:00 ss1_2.sh
  31720 pts/1    00:00:00 sleep
  31748 pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  31157 pts/1    00:00:00 bash
  31719 pts/1    00:00:00 ss1_2.sh
  31720 pts/1    00:00:00 sleep
  31759 pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ fg %1
./ss1_2.sh
ps
ps
^Z
[1]+  Stopped                 ./ss1_2.sh
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ bg %1
[1]+ ./ss1_2.sh &
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ps
    PID TTY          TIME CMD
  31157 pts/1    00:00:00 bash
  31719 pts/1    00:00:00 ss1_2.sh
  31720 pts/1    00:00:00 sleep
  31870 pts/1    00:00:00 ps
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ jobs
[1]+  Running                 ./ss1_2.sh &
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$
```

First, I use “./ss1_2.sh” to run the program in the foreground and after I ”Ctrl + Z” the program has stopped as seen from “jobs” but not entirely killed. Then, using the “fg” command the program now is running in the foreground and I’m back to not being able to use the terminal, but using “bg” command will run the program in the background similar to using “&” and the terminal is still usable. I can use “jobs” to identify the program is running. **In conclusion**, “fg” continues the program in the foreground, and “bg” in the background.

3) Find and try out process monitoring tools with GUI on Linux system



1) What is it?

Resources is a graphical system monitoring application included with Ubuntu. It allows users to monitor running processes and system resource usage.

2) How to acquire it?

“Resources” is already included with the Ubuntu installation.

3) How to use it?

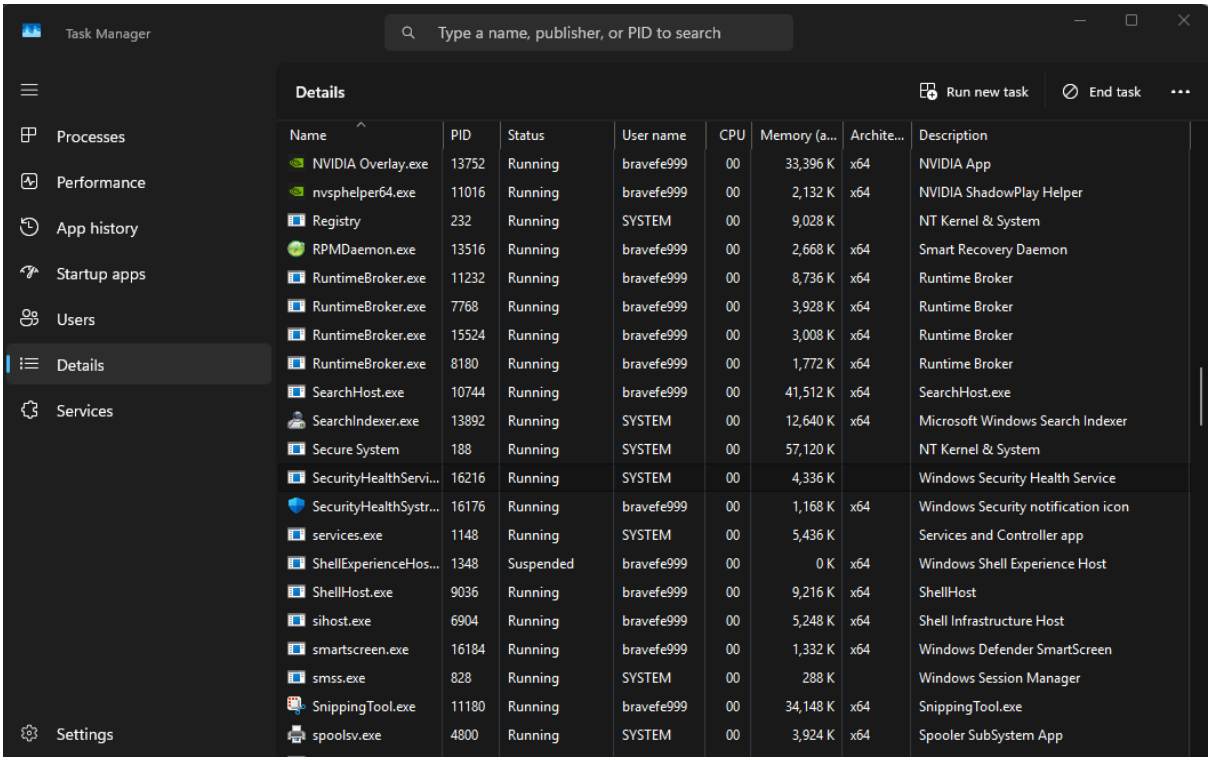
You need to type command “resources” or click on the app

4) What value can it monitor?

- CPU usage

- Memory/RAM usage
- Disk usage/activity
- Network activity
- Running processes
- Process ID (PID)
- User

4) Find and try out process monitoring tools on Windows system



1) What is it?
Windows Task Manager is a built-in Windows utility used to monitor running processes and system resource usage.

2) How to acquire it?
Task Manager is already included with the Window installation.

3) How to use it?
“Ctrl + Shift + Esc” or click on task manager app

4) What value can it monitor?
- CPU usage
- Memory/RAM usage
- Disk usage/activity
- Network activity
- Running processes
- Process ID (PID)
- User
- Architecture
- Description

5) Write C function to simulate the **STCF**(Shortest Time-to-Completion First), the program ‘PS3.c’ that is already provided will read the data in .csv format and convert it into a struct, before sending it to your function, which must process this struct and show which process is running for each time slot.

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./PS3 case1.csv
PID=1 | Arrival=0 | Burst=7 | Remaining=7
PID=2 | Arrival=2 | Burst=4 | Remaining=4
PID=3 | Arrival=4 | Burst=1 | Remaining=1

Time 0: PID=1 | Remaining=7
Time 1: PID=1 | Remaining=6
Time 2: PID=2 | Remaining=4
Time 3: PID=2 | Remaining=3
Time 4: PID=3 | Remaining=1 | DONE at 5
Time 5: PID=2 | Remaining=2
Time 6: PID=2 | Remaining=1 | DONE at 7
Time 7: PID=1 | Remaining=5
Time 8: PID=1 | Remaining=4
Time 9: PID=1 | Remaining=3
Time 10: PID=1 | Remaining=2
Time 11: PID=1 | Remaining=1 | DONE at 12

Total time: 12
Processes completed: 3/3
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./PS3 case2.csv
PID=1 | Arrival=0 | Burst=2 | Remaining=2
PID=2 | Arrival=5 | Burst=3 | Remaining=3
PID=3 | Arrival=5 | Burst=3 | Remaining=3
PID=4 | Arrival=6 | Burst=1 | Remaining=1

Time 0: PID=1 | Remaining=2
Time 1: PID=1 | Remaining=1 | DONE at 2
Time 2: IDLE
Time 3: IDLE
Time 4: IDLE
Time 5: PID=2 | Remaining=3
Time 6: PID=4 | Remaining=1 | DONE at 7
Time 7: PID=2 | Remaining=2
Time 8: PID=2 | Remaining=1 | DONE at 9
Time 9: PID=3 | Remaining=3
Time 10: PID=3 | Remaining=2
Time 11: PID=3 | Remaining=1 | DONE at 12

Total time: 12
Processes completed: 4/4
```

```
pai@pai-H610M-H-DDR4:/run/media/pai/4CE2BF8FE2BF7BB0/_Disk/KMUTT/Year 3/cpe333_lab01_operating_system/Lab 2$ ./PS3 case3.csv
PID=2 | Arrival=0 | Burst=5 | Remaining=5
PID=1 | Arrival=3 | Burst=2 | Remaining=2
PID=3 | Arrival=6 | Burst=1 | Remaining=1

Time 0: PID=2 | Remaining=5
Time 1: PID=2 | Remaining=4
Time 2: PID=2 | Remaining=3
Time 3: PID=1 | Remaining=2
Time 4: PID=1 | Remaining=1 | DONE at 5
Time 5: PID=2 | Remaining=2
Time 6: PID=2 | Remaining=1 | DONE at 7
Time 7: PID=3 | Remaining=1 | DONE at 8

Total time: 8
Processes completed: 3/3
```

Case 1

- P1 runs T0–2 remaining 7->5
- P2 arrives at 2 with remaining=4 < P1 remaining=5. Therefore, run P2 at T2-4 remaining 4->2
- P3 arrives at 4 with remaining=1 < P2 remaining=2. Therefore, run P3 at T4-5 remaining 1->0 and finish at 5.
- Since P2 remaining=2 < P1 remaining=5, run P2 at T5-7 T2->0 and finish at 7.
- Run P1 T7-12 remaining 5->0 and finish at 12.

Case 2

- P1 runs T0–2, remaining 2->0 and finishes at T2.
- Have Nothing to do at T2–5.
- P2 and P3 arrive at T5 P2 remaining=3 == P3 remaining=3 tie-breaker PID 2<3. Therefore, run P2 at T5–6, remaining 3->2.
- P4 arrives at T6 with remaining=1 < P2 remaining=2. Therefore, run P4 at T6–7, remaining 1->0 and finish at T7.
- Run P2 at T7–9, remaining 2->0 and finish at T9.
- Run P3 at T9–12, remaining 3->0 and finish at T12.

Case 3

- P2 runs T0–3, remaining 5->2.
- P1 arrives at T3 with remaining=2 == P2 remaining=2 tie-breaker PID 1<2. Therefore, run P1 at T3–5, remaining 2->0 and finish at T5.
- Run P2 at T5–6, remaining 2->0 and finish at T7.
- P3 arrives at T6 with remaining=1 == P2 remaining=1 tie-breaker PID 2<3. Therefore, run P2 at T6-7, remaining 1->0 and finish at T7.
- Run P3 at T7–8, remaining 1->0 and finish at T8.

Assume:

- Your function and context switch occur very fast. You shouldn't count this as CPU usage.
- The process that arrives at time T means "arrives at the very end of time T-1, but before the scheduler updates the ready queue for time T"
- If multiple processes arrive at the same time, sort by using process_id.'
- If multiple processes have the same remaining time, sort by using process_id.'

All of the input cases and the code that processes them are already in LEB2. You may use the PS3.c and fill in the provided blank space in the function with your version of the scheduler.